# Picasso: Graph Coloring on the Cerebras WSE

An End-to-End Walkthrough

Siddharth Banakar

## Contents

**1 System Architecture Overview** **3**
1.1 File Map . . . 3
1.2 The Running Example . . . 3

**2 Stage 1: Pauli Commutativity → Conflict Graph** **3**
2.1 The Commutativity Rule . . . 3
2.2 Worked Example: $v_0$ = IIXIIIII vs. $v_3$ = IXIIIIII . . . 4
2.3 Result: Dense Conflict Graph . . . 4

**3 Stage 2: CSR Construction** **4**
3.1 From Edge List to CSR . . . 4
3.2 Concrete Arrays for Our 12-Node Graph . . . 5
3.3 Memory Cost . . . 5

**4 Stage 3: Graph Partitioning Across PEs** **5**
4.1 Vertex Assignment: Hash-Based Partitioning . . . 6
4.2 Local CSR Construction . . . 6
4.3 Boundary Table Construction . . . 6
4.4 Stateless GID-Based Routing . . . 7
4.5 4-PE Partitioning . . . 7

**5 Stage 4: Compilation and Fabric Wiring** **8**
5.1 Compilation Command . . . 8
5.2 Fabric Layout (`layout.csl`) . . . 8
5.3 Per-PE Code Instantiation . . . 9

**6 Stage 5: Host-to-Device Data Transfer** **9**
6.1 SDK Initialization . . . 9
6.2 Per-PE Upload Loop . . . 9
6.3 Launch . . . 10

**7 Stage 6: On-Fabric Execution — The PE Kernel** **11**
7.1 Entry Point: `start_coloring()` . . . 11
7.2 Phase 1: Speculate . . . 11
7.3 Phase 2: Send Boundary Wavelets . . . 12
7.3.1 Wavelet Format . . . 12
7.3.2 Cooperative Send Chain . . . 13

7.3.3 Completion Detection . . . 14
7.4 Phase 3: Receive and Relay . . . 15
7.5 Global Barrier with Convergence Detection . . . 16
7.5.1 Barrier Correctness: Blocked Task Gating . . . 16
7.5.2 Round Boundary Safety . . . 17
7.6 Phase 4: Detect and Resolve Boundary Conflicts . . . 18
7.7 Round-by-Round Trace (2 PEs) . . . 19

**8 Stage 7: Device-to-Host Result Retrieval 19**
8.1 Reading Colors . . . 19
8.2 Hardware Cycle Counter Readback . . . 20

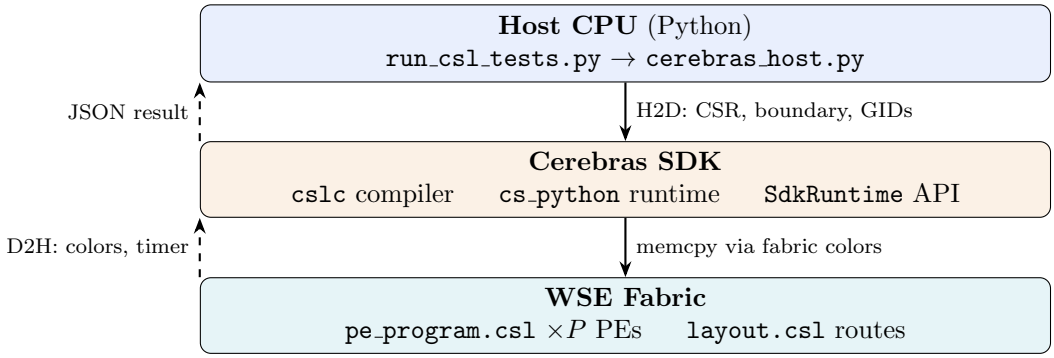
**9 Stage 8: Validation 21**
9.1 Correctness: No Adjacent Vertices Share a Color . . . 21
9.2 Quality: Comparison with Golden Reference . . . 21

**10 Design Rationale: Why Cooperative One-at-a-Time Send 21**
10.1 Iteration 1: Bulk Async Broadcast (Adjacent-Only, 1D) . . . 22
10.2 The Backpressure Problem . . . 22
10.3 Iteration 2: 2D Extension (Same Problem, Worse) . . . 23
10.4 Iteration 3: Sync Barrier (Addressed Timing, Not Backpressure) . . . 23
10.5 V1–V3 Wavelet Format and Routing . . . 23
10.6 Iteration 4: The Cooperative Solution (Current) . . . 24
10.7 Summary of Design Evolution . . . 27

**11 Scaling to WSE-3: Limitations and Path Forward 28**
11.1 The Root Cause: Hash Partitioning Destroys Locality . . . 28
11.2 Priority 0 — Will Not Compile at Scale . . . 28
11.3 Priority 1 — Will Compile but Produce Wrong Results . . . 29
11.4 Priority 2 — Performance Bottlenecks . . . 29
11.5 Priority 3 — Host-Side Preprocessing . . . 30
11.6 SRAM Budget After Locality-Aware Fixes . . . 30
11.7 Recommended Implementation Order . . . 31

# 1 System Architecture Overview

The implementation has three layers, each with a clear job:




## 1.1 File Map

| File | Language | Role |
|---|---|---|
| `picasso/run_csl_tests.py` | Python | Test orchestrator: loads Paulis, builds CSR, partitions, compiles, ru |
| `picasso/cerebras_host.py` | Python | Runs inside `cs_python` container; uploads data, launches kernel |
| `csl/layout.csl` | CSL | Fabric layout: PE grid, color routes, sync barrier wiring |
| `csl/pe_program.csl` | CSL | PE kernel: speculate, send, recv, barrier, detect/resolve |
| `tests/inputs/*.json` | JSON | Pauli string test inputs |

## 1.2 The Running Example

Throughout this document we follow a concrete example: the 12-vertex graph from `test6_12nodes.json`. The raw data is a set of 8-qubit Pauli strings from a molecular Hamiltonian fragment:

| ID | Pauli String | ID | Pauli String |
|---|---|---|---|
| $v_0$ | IIXIIIII | $v_6$ | XIIIIIII |
| $v_1$ | IIYIIIII | $v_7$ | XXIIIIII |
| $v_2$ | IIZIIIII | $v_8$ | YIIIIIII |
| $v_3$ | IXIIIIII | $v_9$ | YYIIIIII |
| $v_4$ | IYIIIIII | $v_{10}$ | ZIIIIIII |
| $v_5$ | IZIIIIII | $v_{11}$ | ZZIIIIII |

The resulting graph has 12 vertices, 45 edges, and an average degree of 7.5. Most vertices are neighbors of most others, so this is far from a toy problem.

# 2 Stage 1: Pauli Commutativity → Conflict Graph

## 2.1 The Commutativity Rule

Two Pauli strings *commute* when the number of qubit positions where both are non-identity and different is **even**; otherwise they *anti-commute*. Commuting pairs become edges in our conflict graph—they can share a color group, so the coloring algorithm needs to keep track of them.

Listing 1: Commutativity check (from `run_csl_tests.py`)

```
def is_an_edge(s1, s2):
    """True if two Pauli strings anti-commute."""
    count = 0
    for c1, c2 in zip(s1, s2):
        if c1 != 'I' and c2 != 'I' and c1 != c2:
            count += 1
    return count % 2 == 1
```

### 2.2 Worked Example: $v_0$ = IIXIIIII vs. $v_3$ = IXIIIIII

| Q7 | Q6 | Q5 | Q4 | Q3 | Q2 | Q1 | Q0 | |
|---|---|---|---|---|---|---|---|---|
| I | I | X | I | I | I | I | I | $v_0$ |
| I | X | I | I | I | I | I | I | $v_3$ |
| ✓ | – | – | ✓ | ✓ | ✓ | ✓ | ✓ | |

Position Q6: $v_0$=I (identity, skip). Position Q5: $v_0$=X, $v_3$=I (one identity, skip). No positions have both non-identity *and* different. Count = 0 (even) ⇒ they **commute** ⇒ edge $(v_0, v_3)$ exists.

### 2.3 Result: Dense Conflict Graph

The $\binom{12}{2} = 66$ pairs yield **45 commuting pairs** (edges) and 21 anti-commuting pairs (no edge). The resulting graph is dense: average degree $= 2 \times 45/12 = 7.5$.

Listing 2: Graph construction (from `run_csl_tests.py`)

```
def build_conflict_graph(paulis):
    n = len(paulis)
    edges = []
    num_commuting = 0
    for i in range(n - 1):
        for j in range(i + 1, n):
            if not is_an_edge(paulis[i], paulis[j]):
                edges.append((i, j))
                num_commuting += 1
    return n, edges, num_commuting
```

## 3 Stage 2: CSR Construction

### 3.1 From Edge List to CSR

The 45 undirected edges become 90 directed arcs (one in each direction) and get packed into two flat arrays—`offsets[13]` and `adj[90]`:

Listing 3: CSR builder (from `run_csl_tests.py`)

```
def build_csr(num_verts, edges):
    from collections import defaultdict
    adj_list = defaultdict(list)
    for u, v in edges:
```

```
        adj_list[u].append(v)
        adj_list[v].append(u)        # symmetric
    offsets = [0]
    adj = []
    for v in range(num_verts):
        neighbors = sorted(adj_list[v])
        adj.extend(neighbors)
        offsets.append(len(adj))
    return np.array(offsets, dtype=np.int32), np.array(adj, dtype=np.int32)
```

### 3.2 Concrete Arrays for Our 12-Node Graph

Listing 4: CSR arrays for test6_12nodes

```
offsets = [0, 9, 18, 27, 34, 41, 48, 55, 62, 69, 76, 83, 90]

adj = [3,4,5,6,7,8,9,10,11,    // v0:  9 neighbors
       3,4,5,6,7,8,9,10,11,    // v1:  9 neighbors
       3,4,5,6,7,8,9,10,11,    // v2:  9 neighbors
       0,1,2,6,7,8,10,         // v3:  7 neighbors
       0,1,2,6,8,9,10,         // v4:  7 neighbors
       0,1,2,6,8,10,11,        // v5:  7 neighbors
       0,1,2,3,4,5,7,          // v6:  7 neighbors
       0,1,2,3,6,9,11,         // v7:  7 neighbors
       0,1,2,3,4,5,9,          // v8:  7 neighbors
       0,1,2,4,7,8,11,         // v9:  7 neighbors
       0,1,2,3,4,5,11,         // v10:  7 neighbors
       0,1,2,5,7,9,10]         // v11:  7 neighbors
```

**Lookup example: neighbors of $v_6$.**

$$\texttt{offsets}[6] = 48, \quad \texttt{offsets}[7] = 55 \quad \Longrightarrow \quad \texttt{adj}[48..55) = [0, 1, 2, 3, 4, 5, 7]$$

Seven neighbors, found with two integer lookups and one contiguous scan.

### 3.3 Memory Cost

| Array | Size (bytes) |
|---|---|
| `offsets[13]` | $13 \times 4 = 52$ |
| `adj[90]` | $90 \times 4 = 360$ |
| **Total CSR** | 412 bytes |

For comparison, a $12 \times 12$ adjacency matrix would need 144 bytes (booleans only), but CSR wins hands-down for larger sparse graphs and avoids pointer overhead entirely.

## 4 Stage 3: Graph Partitioning Across PEs

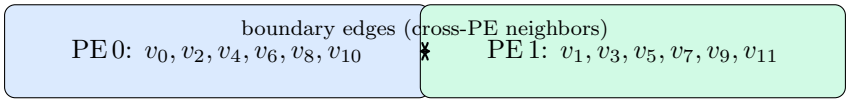
This is the critical bridge between the host CPU and the Cerebras fabric. The partitioner splits the graph so that each PE gets:

- A **local CSR** with only its own vertices and all their edges
- A **boundary table** listing every edge that crosses to another PE, plus routing info

### 4.1 Vertex Assignment: Hash-Based Partitioning

Vertices go to PEs using a simple **hash**: `pe` = `gid` & (`total_pes` − 1). The PE count must be a power of 2. This distributes vertices evenly and avoids the load imbalance you get with contiguous chunking when high-degree hubs cluster in one partition.

With 2 PEs (1D layout), vertex $v_i$ maps to PE = $i$ & 1—even-numbered vertices go to PE 0, odd to PE 1:




Listing 5: Hash-based vertex-to-PE assignment (from `run_csl_tests.py`)

```
total_pes = num_cols * num_rows
assert total_pes & (total_pes - 1) == 0  # must be power of 2
pe_mask = total_pes - 1

def gid_to_pe(gid):
    return gid & pe_mask
# 2 PEs: v0,v2,v4,v6,v8,v10 -> PE 0;  v1,v3,v5,v7,v9,v11 -> PE 1
```

### 4.2 Local CSR Construction

Each PE gets a *local* CSR that contains **all** edges of its vertices—including edges to remote vertices:

Listing 6: PE 0's local CSR (6 vertices, 46 adjacency entries)

```
PE 0 local_offsets = [0, 9, 18, 25, 32, 39, 46]
PE 0 local_adj = [3,4,5,6,7,8,9,10,11,  // v0  (global id 0): 9 neighbors
                  3,4,5,6,7,8,9,10,11,  // v2  (global id 2): 9 neighbors
                  0,1,2,6,8,9,10,        // v4  (global id 4): 7 neighbors
                  0,1,2,3,4,5,7,         // v6  (global id 6): 7 neighbors
                  0,1,2,3,4,5,9,         // v8  (global id 8): 7 neighbors
                  0,1,2,3,4,5,11]        // v10 (global id 10): 7 neighbors
```

Notice that the adjacency array stores *global* vertex IDs (not PE-local indices). This allows the PE kernel to match incoming wavelets by global ID. Each PE also receives a `global_vertex_ids[]` array mapping local index → global ID, and a `pe_mask` value for computing `dest_pe` = `gid`&`pe_mask` at runtime.

### 4.3 Boundary Table Construction

For every edge where the neighbor lives on a different PE, the partitioner records three fields:

| Field | Meaning |
|---|---|
| `boundary_local_idx[b]` | Which local vertex owns this boundary edge |
| `boundary_neighbor_gid[b]` | The remote vertex's global ID |
| `boundary_direction[b]` | Initial send direction: 0=E, 1=W, 2=S, 3=N |

**PE 0's boundary table (24 entries):** Every cross-PE neighbor is on PE 1 (to the east), so all 24 entries have `direction = 0` (east).

Listing 7: PE 0 boundary: first 5 entries (all for $v_0$)

```
boundary_local_idx  = [0, 0, 0, 0, 0, ...]  // v0 has 5 remote neighbors
boundary_neighbor_gid = [3, 5, 7, 9, 11, ...]
boundary_direction  = [0, 0, 0, 0, 0, ...]   // all east
```

**PE 1's boundary table (24 entries):** All cross-PE neighbors are on PE 0 (to the west):

```
boundary_direction  = [1, 1, 1, 1, ...]   // all west
```

### 4.4 Stateless GID-Based Routing

Routing is fully stateless: each relay PE computes the destination PE from the neighbor GID at runtime using the hash function:

$$\texttt{dest_pe} = \texttt{gid} \,\&\, \texttt{pe_mask} \qquad (\texttt{pe_mask} = \texttt{total_pes} - 1)$$

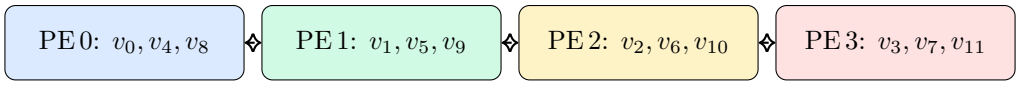
When a wavelet arrives at a relay PE that is not the destination, the PE inspects `dest_pe` and forwards the wavelet in the appropriate direction:

- **1D layout:** compare `dest_pe` with `pe_id`; relay east or west accordingly.
- **2D layout:** Manhattan horizontal-first routing. The relay PE decomposes `dest_pe` into `(dest_col, dest_row)` via repeated subtraction (WSE has no general integer divide), routes horizontally until the column matches, then vertically.

This approach scales to arbitrary grid sizes without any host-side hop precomputation.

### 4.5 4-PE Partitioning

With 4 PEs (hash partitioning: $\texttt{pe} = \texttt{gid} \,\&\, 3$), each PE gets 3 vertices:




The hash-based mapping interleaves vertices across PEs instead of assigning contiguous blocks. This spreads high-degree vertices more evenly. Boundary edges may cross multiple PEs: PE 0→PE 2 needs one relay through PE 1; PE 0→PE 3 needs two relays. Each relay PE figures out the next hop at runtime using $\texttt{dest_pe} = \texttt{gid} \,\&\, \texttt{pe_mask}$—no host-precomputed hop counts needed.

# 5 Stage 4: Compilation and Fabric Wiring

## 5.1 Compilation Command

The test runner invokes `cslc` with compile-time parameters that size all PE-local arrays:

Listing 8: Compilation command for 2-PE layout

```
cslc csl/layout.csl \
  --fabric-dims=10,3 --fabric-offsets=4,1 \
  --memcpy --channels=1 \
  -o csl_compiled_out \
  --params=num_cols:2,num_rows:1,\
    max_local_verts:6,max_local_edges:46,\
    max_boundary:24,max_relay:1,\
    palette_size:16
```

**Key sizing parameters:**

- `max_local_verts=6`: the maximum number of vertices any PE holds (used to size `colors[6]`, `tentative[6]`, `csr_offsets[7]`, etc.)
- `max_local_edges=46`: sizes `csr_adj[46]`
- `max_boundary=24`: sizes all three boundary arrays
- `max_relay=1`: relay buffer depth per direction (for 2 PEs, no relay needed)
- `palette_size=16`: sizes the `forbidden[16]` bitmap

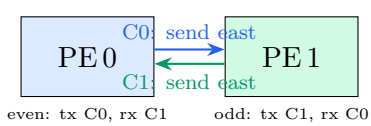
These are the *max across all PEs across all tests*, so one compiled binary handles any test in the suite.

## 5.2 Fabric Layout (`layout.csl`)

The layout file creates a `num_cols` × `num_rows` PE grid and wires 11 fabric colors:

| Colors | Purpose | Routing |
|---|---|---|
| C0–C1 | East data wavelets | Checkerboard: even PEs send on C0, odd on C1 |
| C2–C3 | West data wavelets | Same checkerboard, opposite direction |
| C4–C5 | South data (2D only) | Checkerboard vertical |
| C6–C7 | North data (2D only) | Checkerboard vertical |
| C8–C9 | Sync row-reduce (1D) | East→west chain, alternating colors |
| C10 | Sync broadcast (1D) | West→east from P(0,0) |

**Checkerboard routing.** Adjacent PEs cannot both send on the same fabric color at the same time (hardware rule). The fix: even-column PEs send eastward on C0 and receive on C1; odd-column PEs do the reverse. No two adjacent PEs ever fight for the same wire:



### 5.3 Per-PE Code Instantiation

Each PE tile gets an identical copy of `pe_program.csl`, but with unique compile-time parameters:

Listing 9: PE instantiation in `layout.csl` (simplified)

```
@set_tile_code(col, row, "pe_program.csl", .{
    .pe_id           = pe_idx,
    .num_cols        = num_cols,
    .is_west_edge    = (col == 0),
    .is_east_edge    = (col == num_cols - 1),
    .send_east_color = if (parity==0) east_color_0 else east_color_1,
    .recv_east_color = if (parity==0) west_color_0 else west_color_1,
    // ... 18 more parameters
});
```

Edge flags (`is_west_edge`, `is_east_edge`) let boundary PEs skip sending in directions with no neighbor—a compile-time optimization that removes dead code entirely.

## 6 Stage 5: Host-to-Device Data Transfer

The host script (`cerebras_host.py`) runs inside the Cerebras `cs_python` Singularity container. It uses the SDK's `SdkRuntime` API to upload the partitioned graph.

### 6.1 SDK Initialization

Listing 10: SDK setup (from `cerebras_host.py`)

```
runner = SdkRuntime(compiled_dir, cmaddr=None)

sym_offsets = runner.get_id('csr_offsets')
sym_adj     = runner.get_id('csr_adj')
sym_colors  = runner.get_id('colors')
sym_nverts  = runner.get_id('local_num_verts')
sym_gids    = runner.get_id('global_vertex_ids')
sym_bnd_local = runner.get_id('boundary_local_idx')
# ... boundary_neighbor_gid, boundary_direction,
#     num_boundary, expected_data_recv, expected_done_recv

runner.load()   # Load compiled binary onto fabric
runner.run()    # Start memcpy infrastructure
```

Each `get_id()` call resolves a symbol name (matching `@export_name` in `layout.csl`) to a handle used for memory transfers.

### 6.2 Per-PE Upload Loop

For each PE, the host pads arrays to `max_local_verts`/`max_local_edges`/`max_boundary` (required by the fixed compile-time array sizes) and transfers them:

Listing 11: Uploading to one PE (from `cerebras_host.py`)

```
for pe_idx in range(total_pes):
```

```
    pe_col = pe_idx % num_cols
    pe_row = pe_idx // num_cols
    d = graph_data[pe_idx]

    # Pad to compile-time max sizes
    off_padded = np.zeros(max_local_verts + 1, dtype=np.int32)
    off_padded[:len(d['local_offsets'])] = d['local_offsets']

    adj_padded = np.zeros(max_local_edges, dtype=np.int32)
    adj_padded[:len(d['local_adj'])] = d['local_adj']

    # Transfer via memcpy (DMA through fabric)
    runner.memcpy_h2d(sym_offsets, off_padded,
                      pe_col, pe_row, 1, 1,
                      max_local_verts + 1, streaming=False,
                      data_type=MemcpyDataType.MEMCPY_32BIT,
                      nonblock=False)
    # ... repeat for adj, colors, gids, boundary arrays
```

**Transfer breakdown for our 2-PE example:**

| Symbol | Words/PE | × 2 PEs | Total |
|---|---|---|---|
| csr_offsets | 7 | 14 | 56 B |
| csr_adj | 46 | 92 | 368 B |
| colors | 6 | 12 | 48 B |
| local_num_verts | 1 | 2 | 8 B |
| global_vertex_ids | 6 | 12 | 48 B |
| boundary_local_idx | 24 | 48 | 192 B |
| boundary_neighbor_gid | 24 | 48 | 192 B |
| boundary_direction | 24 | 48 | 192 B |
| num_boundary | 1 | 2 | 8 B |
| pe_mask | 1 | 2 | 8 B |
| expected_data_recv | 1 | 2 | 8 B |
| expected_done_recv | 1 | 2 | 8 B |
| **Total uploaded** | | | **1,136 B** |

## 6.3 Launch

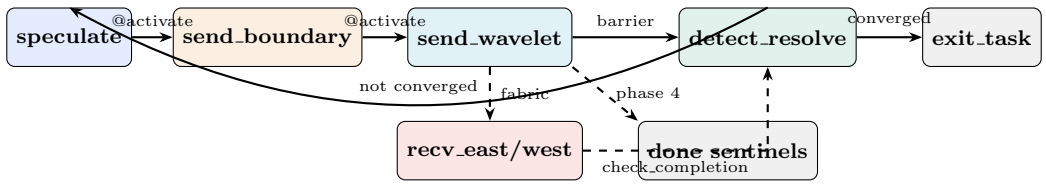
After all data is uploaded, one host call launches the kernel on every PE at once:

```
runner.launch('start_coloring', nonblock=False)
```

This resolves to `start_coloring()` exported from `pe_program.csl`. From this point on, **no host involvement occurs** until all rounds finish—the PEs run entirely on their own.

# 7 Stage 6: On-Fabric Execution — The PE Kernel

This is the heart of the whole system. Every PE runs the same task-driven program, chained in a BSP loop: Speculate → Send → Recv → Barrier → Detect/Resolve → (loop or converge). There is no fixed round limit; the algorithm keeps going until **global convergence** is detected through an OR-reduce piggybacked on the barrier (Section 7.5).




## 7.1 Entry Point: `start_coloring()`

Listing 12: Entry point (from `pe_program.csl`)

```
fn start_coloring() void {
    start_timing();       // enable hardware TSC counter

    // Initialize remote color buffer to "unknown"
    var b: i16 = 0;
    while (b < max_boundary) : (b += 1) {
        remote_recv_color[b] = -1;
    }
    current_round = 0;
    data_recv_remaining = expected_data_recv[0];
    done_recv_count = 0;   // done sentinels received this round
    // ... reset all state flags (local_send_done, phase_complete, etc.) ...

    @activate(speculate_task_id);  // kick off first round
}
```

## 7.2 Phase 1: Speculate

Each PE iterates over its local vertices and assigns the smallest available color:

Listing 13: Speculate task (simplified from `pe_program.csl`)

```
task speculate() void {
    const n = local_num_verts[0];
    var v: i16 = 0;
    while (v < max_local_verts) : (v += 1) {
        if (v >= n) break;
        if (colors[v] >= 0) continue;  // already colored

        // Build forbidden set from ALL neighbors
        // (local: direct lookup, remote: from last round's wavelets)
        var c: i16 = 0;
        while (c < palette_size) : (c += 1) { forbidden[c] = 0; }

        const start = csr_offsets[v];
```

```
        const end_idx = csr_offsets[v + 1];
        var e: i32 = start;
        while (e < end_idx) : (e += 1) {
            const neighbor_gid = csr_adj[e];
            const local_idx = global_to_local(neighbor_gid);
            if (local_idx >= 0) {
                // Local neighbor: read directly
                const nc = colors[local_idx];
                if (nc >= 0) forbidden[nc] = 1;
            } else {
                // Remote neighbor: check last round's wavelet data
                // (stored in remote_recv_color[])
                // ... scan boundary table for matching GID ...
            }
        }
        tentative[v] = find_min_available_color();
        if (tentative[v] >= 0) colors[v] = tentative[v];
    }

    // Local conflict check: higher GID yields
    // ... scan local edges for tentative[v] == tentative[w] ...

    @activate(send_boundary_task_id);
}
```

**Round 1 trace for PE 0 (2-PE layout):** With hash partitioning, PE 0 holds $\{v_0, v_2, v_4, v_6, v_8, v_{10}\}$. Since `colors[v]` is set immediately during the forward scan, later vertices see colors assigned to earlier local vertices in the same speculate call:

| Vertex | Forbidden (from local neighbors) | Chosen | Notes |
|---|---|---|---|
| $v_0$ | $\{\}$ (local nbrs $v_4, v_6, v_8, v_{10}$ uncolored) | color 0 | First available |
| $v_2$ | $\{\}$ (local nbrs $v_4, v_6, v_8, v_{10}$ uncolored; $v_0$ not adjacent) | color 0 | $v_0 \notin \text{adj}(v_2)$ |
| $v_4$ | $\{0\}$ ($v_0$=0, $v_2$=0 are local nbrs) | color 1 | Avoids 0 |
| $v_6$ | $\{0, 1\}$ ($v_0$=0, $v_2$=0, $v_4$=1) | color 2 | |
| $v_8$ | $\{0, 1\}$ ($v_0$=0, $v_2$=0, $v_4$=1) | color 2 | Same as $v_6$ |
| $v_{10}$ | $\{0, 1\}$ ($v_0$=0, $v_2$=0, $v_4$=1) | color 2 | Same again |

**Local conflict check:**

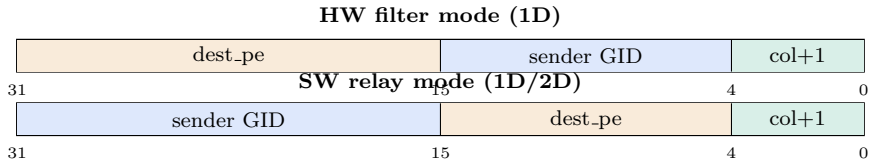
- $v_0$=0 vs $v_2$=0: **not adjacent**—$v_0$ and $v_2$ are not in each other's neighbor lists, so no conflict
- $v_6$=2, $v_8$=2, $v_{10}$=2: pairwise **not adjacent**—no conflicts

After local conflict resolution: **PE 0 keeps all 6 colors**—$v_0$=0, $v_2$=0, $v_4$=1, $v_6$=2, $v_8$=2, $v_{10}$=2.

## 7.3 Phase 2: Send Boundary Wavelets

### 7.3.1 Wavelet Format

Each wavelet is a single 32-bit word packing the sender GID, destination PE, and color into a compact encoding. Two compile-time modes exist, selected by the `use_hw_filter` flag:




Listing 14: Wavelet packing (from `pe_program.csl`)

```
fn pack_wavelet(gid: i32, dest_pe: i32, col: i32) u32 {
  if (use_hw_filter) {
    // HW filter: dest_pe in upper 16 (filter key)
    const upper    = @as(u32, @as(u16, @as(i16, dest_pe))) << 16;
    const gid_part = (@as(u32, @as(u16, @as(i16, gid))) & 0x7FF) << 5;
    const col_part = @as(u32, @as(u16, @as(i16, col + 1))) & 0x1F;
    return upper | gid_part | col_part;
  } else {
    // SW relay: sender_gid in upper 16, dest_pe in [15:5], color in [4:0]
    const id_part  = @as(u32, @as(u16, @as(i16, gid)));
    const dpe_part = (@as(u32, @as(u16, @as(i16, dest_pe))) & 0x7FF) << 5;
    const col_part = @as(u32, @as(u16, @as(i16, col + 1))) & 0x1F;
    return (id_part << 16) | dpe_part | col_part;
  }
}
```

In both modes, the color is stored as $c + 1$ so that color 0 maps to value 1, leaving value 0 free to mean "uncolored." Done sentinels use `color` $= -1$, which encodes as $-1 + 1 = 0$ in the 5-bit color field—this lets us multiplex data and control signals in the same wavelet format with zero overhead.

### 7.3.2 Cooperative Send Chain

Boundary wavelets are sent **one per task activation** to avoid output FIFO deadlock. Between sends, receive data tasks can fire and drain input FIFOs:

Listing 15: One-at-a-time send (from `pe_program.csl`)

```
task send_wavelet() void {
    // Priority: drain one relay wavelet first
    if (drain_one_relay()) {
        @activate(send_wavelet_task_id);
        return;
    }
    // Phase 0: east sends
    if (send_phase == 0) {
        if (!is_east_edge and send_idx < east_count) {
            send_wavelet_east(east_send_buf[send_idx]);
            send_idx += 1;
            @activate(send_wavelet_task_id);
            return;
        }
        send_phase = 1; send_idx = 0;
    }
    // Phase 1: west sends ... Phase 2: south ... Phase 3: north
```

```
    // Phase 4: done sentinels (one per unique destination PE)
    if (send_phase == 4) {
        if (send_idx < done_send_count) {
            const dir = done_send_dir[send_idx];
            const wval = done_send_buf[send_idx];
            if      (dir == 0) send_wavelet_east(wval);
            else if (dir == 1) send_wavelet_west(wval);
            // ... south, north for 2D
            send_idx += 1;
            @activate(send_wavelet_task_id);
            return;
        }
        send_phase = 5; send_idx = 0;
    }
    // Phase 5: final relay drain + completion check
}
```

### 7.3.3 Completion Detection

The host precomputes `expected_data_recv` (the total number of data wavelets each PE should receive) and `expected_done_recv` (the number of unique source PEs sending to it), both uploaded alongside the graph data. Each receive handler decrements `data_recv_remaining`; done sentinels increment `done_recv_count`. Completion uses a dual-path check:

Listing 16: Dual-path completion (from `pe_program.csl`)

```
// Host-precomputed: uploaded via H2D
var expected_data_recv: [1]i32;      // total data wavelets to expect
var expected_done_recv: [1]i32;      // unique source PEs sending to us
var data_recv_remaining: i32 = 0;
var done_recv_count: i32 = 0;

fn check_completion() void {
    if (!phase_complete and local_send_done and all_relay_drained()) {
        if (data_recv_remaining <= 0) {
            // Normal path: all expected data received
            phase_complete = true;
            on_data_complete();
        } else if (done_recv_count >= expected_done_recv[0]) {
            // Overflow recovery: all senders finished, but some
            // data was lost to relay buffer overflow. Proceed
            // anyway so the algorithm doesn't hang forever.
            phase_complete = true;
            on_data_complete();
        }
    }
}
```

Two paths reach completion. The **normal path** fires when `data_recv_remaining` reaches zero—every expected wavelet arrived. The **overflow recovery path** fires when every source PE has sent a done sentinel (`done_recv_count` $\geq$ `expected_done_recv`), meaning all senders have finished even though some relay wavelets were dropped. Without the second path, a PE waiting

for lost wavelets would hang forever.

### 7.4 Phase 3: Receive and Relay

Receive handlers fire as hardware data tasks whenever a wavelet shows up on the PE's input queue. The handler is a thin wrapper around `route_data_wavelet()`, which inspects `dest_pe` to decide whether to consume or relay:

Listing 17: East receive handler (from `pe_program.csl`)

```
task recv_east(wval: u32) void {
    route_data_wavelet(wval);
    check_completion();
}

fn route_data_wavelet(wval: u32) void {
    const dest_pe = unpack_dest_pe(wval);
    if (dest_pe == @as(i32, pe_id)) {
        const recv_col = unpack_color(wval);
        if (recv_col < 0) {
            // Done sentinel: sender finished all its sends
            done_recv_count += 1;
        } else {
            // Normal data wavelet
            process_incoming_wavelet(
                unpack_gid(wval), recv_col);
            data_recv_remaining -= 1;
        }
    } else if (num_rows == 1) {
        // 1D: relay east or west based on dest_pe
        if (dest_pe > @as(i32, pe_id)) buffer_relay_east(wval);
        else                           buffer_relay_west(wval);
    } else {
        // 2D: Manhattan horizontal-first routing
    }
}
```

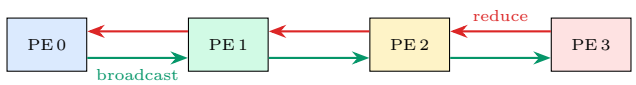
`process_incoming_wavelet()` stores the received color in `remote_recv_color[b]` for every matching boundary entry:

```
fn process_incoming_wavelet(sender_gid: i32, sender_color: i32) void {
    const nbnd = num_boundary[0];
    var b: i16 = 0;
    while (b < max_boundary) : (b += 1) {
        if (@as(i32, b) >= nbnd) break;
        if (boundary_neighbor_gid[b] == sender_gid) {
            remote_recv_color[b] = sender_color;
        }
    }
}
```

## 7.5 Global Barrier with Convergence Detection

Before detect/resolve can run, **every PE must have finished sending and receiving**. In 1D mode, we use a two-phase hardware barrier that also carries convergence info via an OR-reduce:

1. **Row-reduce with OR** (east→west): The easternmost PE sends a sync wavelet west. The wavelet payload is `sync_buf[0]`, which each PE sets to 1 if it has *any* uncolored vertices, or 0 if all local vertices are colored. As each PE forwards the reduce signal west, it **ORs** the incoming value with its own: `sync_buf[0] = sync_buf[0] | wval`. When P(0,0) receives the reduced value, it holds the global OR—1 if *any* PE still has uncolored vertices.
2. **Broadcast** (P(0,0)→all east): P(0,0) sends the reduced value eastward as a broadcast wavelet. Each PE stores this global result.
3. **Convergence check**: When a PE completes the barrier (both data received and broadcast received), it checks: if `current_round > 0` and the reduced value is 0 (no PE has uncolored vertices), the algorithm has **converged** and the PE jumps directly to `exit_task`. Otherwise it proceeds to `detect_resolve`.




A PE only sends its reduce signal **after** `on_data_complete()` fires (meaning all data wavelets have been received). So the reduce signal only ever propagates through PEs that are truly done.

### 7.5.1 Barrier Correctness: Blocked Task Gating

There is a subtle but important invariant that makes this barrier correct: the `sync_reduce_task` starts **blocked** at compile time and only gets unblocked inside `on_data_complete()`. The reduce chain physically *cannot* propagate through a PE until that PE has finished its own data exchange:

Listing 18: Sync tasks start blocked (from `pe_program.csl`)

```
comptime {
    @bind_data_task(sync_reduce_recv, sync_reduce_task_id);
    @bind_data_task(sync_bcast_recv, sync_bcast_task_id);
    @block(sync_reduce_task_id);   // blocked at init
    @block(sync_bcast_task_id);    // blocked at init
}
```

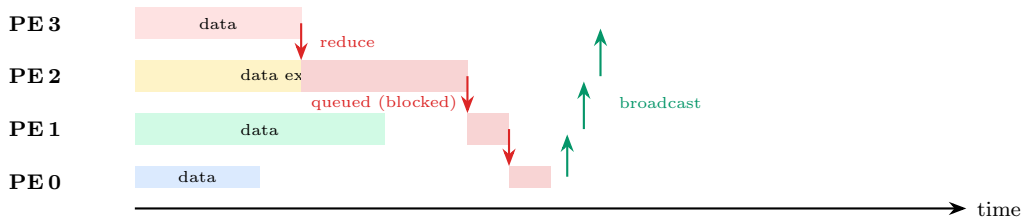
Listing 19: Unblock only when data exchange is complete

```
fn on_data_complete() void {
    local_data_done = true;
    if (is_east_edge) {
        send_sync_reduce();            // east PE initiates chain
        @unblock(sync_bcast_task_id);  // arm for broadcast
    } else {
        @unblock(sync_reduce_task_id); // arm for reduce from east
    }
    maybe_proceed();
}
```

Consider a 4-PE example where PE 3 (eastmost) finishes data exchange first and sends the reduce wavelet west. If PE 2 is still receiving data wavelets:

1. PE 3's reduce wavelet arrives at PE 2's input queue for `sync_reduce_recv_color`
2. PE 2's `sync_reduce_task` is still **blocked** ⇒ the wavelet *sits in the queue without being processed*
3. PE 2 eventually completes data exchange ⇒ `on_data_complete()` calls `@unblock(sync_reduce_task_id)`
4. *Now* PE 2's reduce task fires: it OR-combines and forwards the wavelet westward to PE 1
5. The same gating mechanism applies at PE 1 and PE 0

The broadcast from P(0,0) cannot be sent until P(0,0) receives the completed reduce chain, which requires *every* intermediate PE to have finished and unblocked. Therefore:




**No PE can enter `detect_resolve` until every PE has completed data exchange.** The blocked task mechanism gives us a hardware-enforced gate that makes this guarantee without any polling or busy-waiting.

### 7.5.2 Round Boundary Safety

After the barrier, PEs enter `detect_resolve` and then `speculate` at slightly different times. This is safe because:

1. `maybe_proceed()` calls `block_all_recv()` *before* activating `detect_resolve`—closing the receive window
2. Recv stays blocked through `detect_resolve` → `speculate`
3. Only `send_boundary()` calls `unblock_all_recv()`—*after* resetting `data_recv_remaining`
4. The sync barrier tasks are re-blocked inside their own handlers (`@block(sync_reduce_task_id)`)

Even if a fast PE starts sending round $N+1$ wavelets before a slow PE has entered `detect_resolve` for round $N$, no problem: the slow PE's receive tasks are blocked, so wavelets just queue up harmlessly in the hardware input FIFOs until the PE is ready.

The convergence check in `maybe_proceed()` is:

Listing 20: Convergence-aware barrier completion

```
fn maybe_proceed() void {
    if (local_data_done and sync_barrier_done) {
        // sync_buf[0] holds the global OR-reduced
        // "has_uncolored" flag after the barrier.
        if (current_round > 0 and sync_buf[0] == 0) {
            @activate(exit_task_id);  // converged!
        } else {
            @activate(detect_resolve_task_id);
        }
    }
}
```

The OR-reduce in the sync receive handler:

Listing 21: OR-reduce in barrier receive

```
task sync_reduce_recv(wval: u32) void {
    @block(sync_reduce_task_id);
    sync_buf[0] = sync_buf[0] | wval;  // OR-reduce
    if (is_west_edge) {
        send_sync_bcast();            // broadcast reduced value
        sync_barrier_done = true;
        maybe_proceed();
    } else {
        send_sync_reduce();           // forward west
        @unblock(sync_bcast_task_id); // wait for broadcast
    }
}
```

So there is **no fixed round limit**. The algorithm terminates when every PE's local vertices are colored and no conflicts remain. In practice, the 12-node test converges in just **3 rounds** (one productive round + one conflict-resolution round + one verification round).

### 7.6 Phase 4: Detect and Resolve Boundary Conflicts

Listing 22: Detect/resolve with convergence tracking (from `pe_program.csl`)

```
task detect_resolve() void {
    const nbnd = num_boundary[0];
    var had_conflict: bool = false;
    var b: i16 = 0;
    while (b < max_boundary) : (b += 1) {
        if (b >= nbnd) break;
        const li = boundary_local_idx[b];
        const my_gid = global_vertex_ids[li];
        const nbr_gid = boundary_neighbor_gid[b];
        const my_color = colors[li];
        const remote_color = remote_recv_color[b];
        if (my_color >= 0 and remote_color >= 0
            and my_color == remote_color) {
            if (my_gid > nbr_gid) {
                colors[li] = -1;   // yield: revert to uncolored
                had_conflict = true;
            }
        }
    }
    // Check if any local vertex is still uncolored
    var has_uncolored: bool = had_conflict;
    if (!has_uncolored) {
        const n = local_num_verts[0];
        var v: i16 = 0;
        while (v < max_local_verts) : (v += 1) {
            if (v >= n) break;
            if (colors[v] < 0) { has_uncolored = true; break; }
        }
    }
    // Store in sync_buf for next round's barrier OR-reduce
    sync_buf[0] = if (has_uncolored) 1 else 0;
```

```

    current_round += 1;
    // Reset state and start next round (convergence is
    // detected by the barrier, not by a round counter)
    data_recv_remaining = expected_data_recv[0];  // count-based reset
    done_recv_count = 0;                          // sentinel reset
    // ... reset all relay/sync flags ...
    @activate(speculate_task_id);  // next round
}
```

There is no `max_rounds` check. After each `detect_resolve`, the PE records whether it still has uncolored vertices in `sync_buf[0]`, then starts the next round unconditionally. The *barrier* in the next round reads `sync_buf[0]` via the OR-reduce; if the global OR is 0 (nobody has uncolored vertices), `maybe_proceed()` terminates the algorithm before `detect_resolve` even runs.

### 7.7 Round-by-Round Trace (2 PEs)

**Round 1:**

| | After Speculate | After Local Conflict | After Boundary Resolve |
|---|---|---|---|
| PE 0 | $v_0{=}0, v_2{=}0, v_4{=}1, v_6{=}2, v_8{=}2, v_{10}{=}2$ | all kept (no local conflicts) | all kept |
| PE 1 | $v_1{=}0, v_3{=}1, v_5{=}1, v_7{=}2, v_9{=}1, v_{11}{=}3$ | all kept (no local conflicts) | $v_7, v_9$ yield |

After round 1: 10 vertices colored, 2 remain ($v_7, v_9$ on PE 1). The boundary conflicts are $v_7{=}2$ vs $v_6{=}2$ (GID $7 > 6$, $v_7$ yields) and $v_9{=}1$ vs $v_4{=}1$ (GID $9 > 4$, $v_9$ yields).

**Round 2:** $v_7$ now sees $\{v_1{=}0, v_3{=}1, v_{11}{=}3\}$ (local) and $\{v_0{=}0, v_2{=}0, v_6{=}2\}$ (from wavelets) $\Rightarrow$ forbidden $= \{0, 1, 2, 3\}$, picks color 4. $v_9$ sees $\{v_1{=}0, v_7{=}4, v_{11}{=}3\}$ and $\{v_0{=}0, v_2{=}0, v_4{=}1, v_8{=}2\}$ $\Rightarrow$ forbidden $= \{0, 1, 2, 3, 4\}$, picks color 5. No conflicts remain.

The process converges after **3 rounds** (the third round is a verification round where the barrier's OR-reduce confirms global convergence), using **6 colors**.

## 8 Stage 7: Device-to-Host Result Retrieval

### 8.1 Reading Colors

After `launch()` returns (all PEs have called `sys_mod.unblock_cmd_stream()`), the host reads back each PE's colors:

Listing 23: Color readback (from `cerebras_host.py`)

```
all_colors = np.full(num_verts, -1, dtype=np.int32)
for pe_idx in range(total_pes):
    pe_col = pe_idx % num_cols
    pe_row = pe_idx // num_cols
    d = graph_data[pe_idx]
    local_n = d['local_n']
    buf = np.zeros(max_local_verts, dtype=np.int32)
    runner.memcpy_d2h(buf, sym_colors,
                      pe_col, pe_row, 1, 1,
```

```
                        max_local_verts, streaming=False,
                        data_type=MemcpyDataType.MEMCPY_32BIT)
    # Hash-based: use the global_ids list to map back
    for i in range(local_n):
        gid = d['global_ids'][i]
        all_colors[gid] = buf[i]
```

Colors are reassembled into a global array using each PE's `global_ids` list to map local indices back to global vertex IDs. This mapping is necessary because hash-based partitioning does not give each PE a contiguous range of vertices.

## 8.2 Hardware Cycle Counter Readback

Each PE records start and end timestamps using the WSE hardware TSC (Timestamp Counter). The `<time>` CSL module gives us an always-running cycle counter with sub-cycle precision:

Listing 24: TSC timing (from `pe_program.csl`)

```
const tsc = @import_module("<time>");
var tsc_start_buf = @zeros([tsc.tsc_size_words]u16);
var tsc_end_buf   = @zeros([tsc.tsc_size_words]u16);
var coloring_timer = @zeros([3]f32);

inline fn start_timing() void {
    tsc.enable_tsc();
    tsc.get_timestamp(&tsc_start_buf);
}

inline fn stop_timing() void {
    tsc.get_timestamp(&tsc_end_buf);
    tsc.disable_tsc();

    // Pack 6 x u16 timestamps into 3 x f32 for host readback
    var lo_: u16 = 0; var hi_: u16 = 0;
    lo_ = tsc_start_buf[0]; hi_ = tsc_start_buf[1];
    coloring_timer[0] = @bitcast(f32, (@as(u32, hi_) << @as(u16, 16)) | @as(u32, lo_)
    );
    lo_ = tsc_start_buf[2]; hi_ = tsc_end_buf[0];
    coloring_timer[1] = @bitcast(f32, (@as(u32, hi_) << @as(u16, 16)) | @as(u32, lo_)
    );
    lo_ = tsc_end_buf[1]; hi_ = tsc_end_buf[2];
    coloring_timer[2] = @bitcast(f32, (@as(u32, hi_) << @as(u16, 16)) | @as(u32, lo_)
    );
}
```

The host reads the packed timer data and converts to cycles:

Listing 25: Cycle count computation (from `cerebras_host.py`)

```
from cerebras.sdk.sdk_utils import calculate_cycles

timing_data = np.zeros(num_rows * num_cols * 3, dtype=np.uint32)
runner.memcpy_d2h(timing_data, sym_timer, 0, 0,
                  num_cols, num_rows, 3, ...)
```

```
timing_hwl = timing_data.view(np.float32).reshape(
    (num_rows, num_cols, 3))

max_cycles = 0
for pe_row in range(num_rows):
    for pe_col in range(num_cols):
        cycles = calculate_cycles(
            timing_hwl[pe_row, pe_col, :])
        max_cycles = max(max_cycles, int(cycles))

elapsed_ms = (max_cycles / 850_000_000) * 1000
```

The wall-clock time is the **maximum** across all PEs—the slowest PE sets the total time since the algorithm is synchronous.

# 9 Stage 8: Validation

The test runner checks two things:

## 9.1 Correctness: No Adjacent Vertices Share a Color

Listing 26: Validation (from `run_csl_tests.py`)

```
def validate_coloring(num_verts, edges, colors):
    errors = 0
    for u, v in edges:
        if colors[u] >= 0 and colors[v] >= 0 \
           and colors[u] == colors[v]:
            errors += 1
    uncolored = sum(1 for c in colors if c < 0)
    num_colors = max(colors) + 1
    return errors, uncolored, num_colors
```

Every edge $(u, v)$ is checked: if both endpoints are colored and share the same color, that is a conflict. The algorithm guarantees zero conflicts when it terminates.

## 9.2 Quality: Comparison with Golden Reference

The Cerebras result is compared against the CPU Picasso implementation's golden output:

- **Node/edge counts** must match
- **Color count** should be $\leq$ the reference (our greedy may use fewer colors)

# 10 Design Rationale: Why Cooperative One-at-a-Time Send

The implementation did not start in its present form. We went through four design iterations, each driven by problems we hit in the previous attempt. This section traces that evolution and explains *why* we ended up with the cooperative one-per-activation send chain, relay queues, and count-based completion.

## 10.1 Iteration 1: Bulk Async Broadcast (Adjacent-Only, 1D)

Our first try (`pe_program.csl.1d.bak`) used the simplest possible approach: pack all boundary wavelets into a send buffer, then fire a **single bulk DMA transfer** per direction using `@mov32` with `.async = true`:

Listing 27: Original bulk-send approach (`pe_program.csl.1d.bak`)

```
// Single DMA transfer per direction
var east_out = @get_dsd(fabout_dsd, .{
    .fabric_color = send_east_color,
    .extent = max_boundary,
    .output_queue = send_east_oq,
});
east_out = @set_dsd_length(east_out, east_count);
var east_src = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{max_boundary} -> east_send_buf[i],
});
east_src = @set_dsd_length(east_src, east_count);
@mov32(east_out, east_src, .{ .async = true });
// ... same for west ...
@unblock(recv_east_task_id);  // open receive window
```

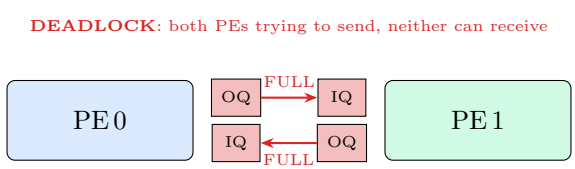
The approach looked clean:

- One `@mov32` per direction—the DMA engine handles the rest
- Barrier via counting: each PE knows how many wavelets to expect
- Only supported adjacent PEs (no multi-hop relay)

## 10.2 The Backpressure Problem

This bulk-send approach had a fatal flaw: **output FIFO backpressure causing circular deadlock**.

On the Cerebras WSE, each PE has small hardware FIFOs (queues) for fabric communication. When PE A fires `@mov32(east_out, ...)` with `async=true`, the DMA engine starts pushing wavelets into the output queue. But these queues are shallow—typically 4–8 words. If the downstream PE B's *input* queue is also full (because PE B is itself blocked trying to *send*), the fabric stalls:




The deadlock scenario:

1. PE 0 starts bulk DMA east → fills its output queue
2. PE 1 simultaneously starts bulk DMA west → fills its output queue
3. PE 0's output stalls because PE 1's input queue is full (PE 1 is busy sending, not draining receives)
4. PE 1's output stalls for the same reason (PE 0 is busy sending, not draining receives)
5. Neither PE can make progress $\Longrightarrow$ **deadlock**

With `@mov32(.async = true)`, the DMA engine takes over the PE—once started, no other tasks (including receive handlers) can interleave until the DMA finishes or yields. When *both* PEs in a pair fire bulk sends at the same time, neither can drain incoming wavelets, and the output FIFOs jam.

This problem gets *worse* with graph density: our 12-node test has 30 boundary edges per PE. Trying to DMA 30 wavelets in one burst guarantees FIFO overflow when the neighbor is doing the same thing.

### 10.3 Iteration 2: 2D Extension (Same Problem, Worse)

The second version (`pe_program.csl.pre-sync.bak`) added 2D grids with north/south support, but **kept the same bulk DMA approach**. With four directions instead of two, things got worse:

Listing 28: Pre-sync 2D version: still bulk DMA per direction

```
@mov32(east_out,  east_src,  .{ .async = true });
@mov32(west_out,  west_src,  .{ .async = true });
@mov32(south_out, south_src, .{ .async = true });   // NEW
@mov32(north_out, north_src, .{ .async = true });   // NEW
```

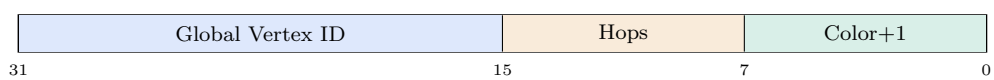
Four simultaneous async DMAs competing for fabric bandwidth, each potentially blocking on a different neighbor's full input queue. The deadlock window grew from two directions to four.

### 10.4 Iteration 3: Sync Barrier (Addressed Timing, Not Backpressure)

The third version (`pe_program.csl.sync`) added the hardware row-reduce/broadcast barrier, correctly solving the *synchronization* problem (making sure all PEs finish sending before detect/resolve runs). But it still used bulk DMA for the sends themselves. The barrier got the sequencing right, but could not prevent the FIFO deadlock during the send phase.

### 10.5 V1–V3 Wavelet Format and Routing

These early versions used a hop-count-based wavelet format with an 8-bit Manhattan hop field:




The boundary table included a fourth field, `boundary_hops[b]`, encoding the Manhattan L-route. The host precomputed horizontal/vertical hop counts so relay PEs could decrement them in-flight:

Listing 29: Hops computation (V1–V3)

```
def make_hops(h_dir, h_hops, v_dir, v_hops):
    return (h_dir << 7) | (h_hops << 4) | (v_dir << 3) | v_hops
```

Listing 30: Wavelet packing (V1–V3)

```
fn pack_wavelet(gid: i32, col: i32, hops: i32) u32 {
    const id_part   = @as(u32, @as(u16, gid));
    const hops_part = @as(u32, @as(u16, hops)) & 0xFF;
    const col_part  = @as(u32, @as(u16, col + 1)) & 0xFF;
```

```
    return (id_part << 16) | (hops_part << 8) | col_part;
}
```

The receive handler had to check for done sentinels, decrement hops, and handle corner turns for 2D routing—far more complex than the current stateless design:

Listing 31: East receive handler (V1–V3)

```
task recv_east(wval: u32) void {
    if (is_done_sentinel(wval)) {
        done_count += 1;
        if (unpack_h_hops(wval) > 0 and !is_west_edge)
            buffer_relay_west(dec_h_hops(wval));
    } else {
        const hh = unpack_h_hops(wval);
        if (hh > 0) {
            buffer_relay_west(dec_h_hops(wval));
        } else if (unpack_v_hops(wval) == 0) {
            process_incoming_wavelet(
                unpack_gid(wval), unpack_color(wval));
        } else {
            // Corner PE: turn from horizontal to vertical
        }
    }
    check_completion();
}
```

Done sentinels used a special GID = `0xFFFF` and were broadcast to all PEs ($O(P^2)$ sentinel traffic system-wide):

Listing 32: Done sentinel (V1–V3 broadcast-style)

```
fn make_done_wavelet(hops: i32) u32 {
    const hops_part = @as(u32, @as(u16, hops)) & 0xFF;
    return @as(u32, 0xFFFF0000) | (hops_part << 8);
}
```

This hop-count approach limited scalability to grid sizes where hop counts fit in 3 bits per axis, required host-side precomputation, and made the receive path complex. All of this was eliminated in V4's stateless GID-based routing.

## 10.6 Iteration 4: The Cooperative Solution (Current)

The current version solves the backpressure problem with three interlocking mechanisms:

**(a) One-per-activation send chain.** Instead of bulk DMA, we send **exactly one wavelet per task activation**. After each send, the task re-activates itself. Between activations, the PE's task scheduler can run pending receive tasks, draining input FIFOs:

Listing 33: Current approach: one wavelet per activation

```
// Send state machine: wavelets are sent one-per-task-activation
// to avoid output FIFO deadlock. Between activations, recv data
// tasks can fire, draining input FIFOs and preventing circular
```

```
// backpressure.
task send_wavelet() void {
    if (drain_one_relay()) {       // priority: relay first
        @activate(send_wavelet_task_id);
        return;
    }
    if (send_phase == 0 and send_idx < east_count) {
        send_wavelet_east(east_send_buf[send_idx]);
        send_idx += 1;
        @activate(send_wavelet_task_id);   // re-arm
        return;
    }
    // ... next phase ...
}
```

The key insight: `@activate()` hands control back to the task scheduler. If a receive task is pending (a wavelet arrived on an input queue), it runs *before* the next `send_wavelet` activation. This interleaving guarantees input FIFOs get drained even while the PE is sending.

**(b) Relay queues decouple receive from send.** When a receive handler gets a wavelet that needs forwarding (multi-hop relay), it does **not** write to the output FIFO directly. Instead, it buffers the wavelet in a software relay queue:

```
// Relay queues: recv handlers buffer wavelets here for
// deferred forwarding. Decouples recv (input) from send
// (output), preventing circular deadlock.
// Implemented as circular ring buffers so drained slots are
// reused within a round.
var relay_east_q: [max_relay]u32;
var relay_east_head: u16 = 0;   // next drain position
var relay_east_tail: u16 = 0;   // next write position
var relay_east_count: u16 = 0;  // items currently buffered
```

Only the `send_wavelet` task reads from relay queues and writes to output FIFOs. This prevents the scenario where a receive handler tries to write to a full output FIFO, which would block it and stop further input queue draining.

**Circular ring buffers.** An earlier version used *linear* append-only relay buffers with separate read (`sent`) and write (`len`) pointers that only advanced forward. Once the `send_wavelet` task drained a slot, that slot was wasted—new arrivals could only append past the end. If more relay wavelets arrived between drain opportunities than the buffer could hold, they were **silently dropped**.

The current implementation uses *circular* ring buffers: drained slots wrap around and become available for new arrivals. A buffer of depth $N$ can now handle unlimited total relay throughput as long as no more than $N$ are in-flight at any instant:

```
fn buffer_relay_east(wval: u32) void {
    if (relay_east_count < max_relay) {
        relay_east_q[relay_east_tail] = wval;
        relay_east_tail += 1;
        if (relay_east_tail >= max_relay) relay_east_tail = 0;
        relay_east_count += 1;
```

```
    }
    @activate(send_wavelet_task_id);  // wake drain
}
fn drain_one_relay() bool {
    if (relay_east_count > 0) {
        send_wavelet_east(relay_east_q[relay_east_head]);
        relay_east_head += 1;
        if (relay_east_head >= max_relay) relay_east_head = 0;
        relay_east_count -= 1;
        return true;
    }
    // ... west, south, north ...
}
```

The buffer depth `max_relay` is computed by the host at compile time. It analyzes every PE's relay load by tracing all boundary wavelets through their Manhattan routes and counting how many pass through each intermediate PE per direction. For example, with our 12-node graph on 4 PEs, the middle PEs (PE 1 and PE 2) each relay up to 20 wavelets per direction per round, so `max_relay=20`. With only 2 PEs (adjacent, no relay needed), `max_relay=1`.

**Relay buffer overflow and the liveness problem.** Even with circular ring buffers, the relay queue has a fixed depth. If wavelets arrive faster than they can be drained—or if a burst of relay traffic exceeds `max_relay` slots—incoming wavelets are **silently dropped**. We track this with a per-PE overflow counter:

```
fn buffer_relay_east(wval: u32) void {
    if (relay_east_count < max_relay) {
        relay_east_q[relay_east_tail] = wval;
        relay_east_tail += 1;
        if (relay_east_tail >= max_relay) relay_east_tail = 0;
        relay_east_count += 1;
    } else {
        perf_counters[7] += 1;   // relay_overflow_drops
    }
    @activate(send_wavelet_task_id);
}
```

The host reads `perf_counters[7]` after each run and reports any drops. But the real hazard is **liveness**: if a data wavelet is dropped, the destination PE waits forever for `data_recv_remaining` to reach zero—it never does, and the whole computation hangs.

**(c) Completion detection: count-based with done sentinel fallback.** The primary completion mechanism is **count-based**: the host precomputes `expected_data_recv` (the total number of data wavelets each PE should receive) and uploads it with the graph data. Each receive handler decrements `data_recv_remaining`; when it hits zero, data exchange is complete.

To guard against relay overflow, every PE also sends **done sentinels**—lightweight wavelets sent to each unique destination PE after all data wavelets have been transmitted. A done sentinel uses the same wavelet format but with `color` $= -1$ (which encodes as $-1 + 1 = 0$ in the 5-bit color field). The host precomputes `expected_done_recv` for each PE: the count of unique source PEs that send data to it.

Listing 34: Done sentinel construction (from `send_boundary`)

```
// Build dedup'd sentinel list: one per unique destination PE
var i: i16 = 0;
while (i < max_boundary) : (i += 1) {
    if (i >= nbnd) break;
    const gid = boundary_neighbor_gid[i];
    const dest = gid & pe_mask[0];
    if (dest != pe_id and done_sent_flags[dest] == 0) {
        done_send_buf[done_send_count] =
            pack_wavelet(pe_id, dest, -1);  // color = -1
        done_send_dir[done_send_count] =
            boundary_direction[i];          // initial direction
        done_send_count += 1;
        done_sent_flags[dest] = 1;
    }
}
```

The done sentinels are sent in **phase 4** of the send chain (after all data phases 0–3), so they always arrive *after* the last data wavelet from that source PE. On the receive side, a sentinel increments `done_recv_count` rather than decrementing `data_recv_remaining`. The dual-path `check_completion()` (shown earlier) uses both counters:

- **Normal path**: `data_recv_remaining` $\leq$ `0` — all data arrived.
- **Overflow recovery**: `done_recv_count` $\geq$ `expected_done_recv` — all senders finished but some data was lost in relay buffers. The PE proceeds with whatever colors it did receive; the missing vertices will be re-colored in the next BSP round.

This guarantees the algorithm makes progress even when relay buffers overflow. In practice, overflow is rare: for our test suite of 9 graphs on 8 PEs, all runs report zero `relay_overflow_drops`. But the sentinel mechanism ensures we never hang if it does happen.

### 10.7 Summary of Design Evolution

| Version | Send Strategy | Barrier | Multi-hop | Problem |
|---|---|---|---|---|
| V1 (1D) | Bulk `@mov32` async | Wavelet counting | No | FIFO deadlock |
| V2 (2D) | Bulk `@mov32` ×4 dirs | Wavelet counting | No | Worse deadlock |
| V3 (sync) | Bulk `@mov32` async | HW row-reduce + broadcast | No | Deadlock still possible |
| V4a | One-per-activation | HW barrier + done sentinels | Yes (relay queues) | Extra sentinel traffic |
| V4b | One-per-activation | HW barrier + count-based | Yes (relay queues) | Hangs on relay overflow |
| **V4c (current)** | **One-per-activation** | **HW barrier + count + sentinels** | **Yes (circular relay)** | **Deadlock-free + overflow-safe** |

The performance cost of one-per-activation sending is higher latency per wavelet (one task switch per wavelet instead of one DMA for the whole batch). But correctness is non-negotiable: a deadlocked program produces no result at all. In practice, the task-switch overhead is roughly 10–20 cycles per wavelet—negligible compared to fabric routing latency. And for large graphs with many boundary edges, the interleaved send/recv pattern actually gets *better* sustained throughput than bulk DMA, because input FIFOs never fill up and fabric backpressure never kicks in.

# 11 Scaling to WSE-3: Limitations and Path Forward

The current implementation runs correctly on small PE grids (up to ∼100 PEs) and handles all test graphs without deadlock or data loss. However, the WSE-3 fabric contains roughly 900,000 processing elements, and several design choices that work fine at small scale become fundamental bottlenecks at that size. This section catalogs the limitations we have identified, traces them to their root cause, and outlines the changes needed to reach full-fabric scale.

## 11.1 The Root Cause: Hash Partitioning Destroys Locality

Before diving into individual issues, it helps to understand why so many of them share a single root cause.

Our current partitioning strategy assigns vertex $v$ to PE $v \,\&\, (total_PEs - 1)$. This is fast and simple, but it completely ignores the graph's structure: two adjacent vertices can end up on opposite corners of the fabric. On a 948×948 grid, that means a single wavelet may travel up to 1,896 relay hops—through nearly every PE on the grid—just to reach its neighbor.

This one decision amplifies ten of the sixteen scaling problems we have cataloged. Replacing hash partitioning with **locality-aware partitioning** (e.g., METIS graph partitioning combined with Hilbert-curve spatial mapping) is therefore the single highest-leverage change. It reduces typical hop distance from $O(grid_dim)$ to $O(1\text{–}5)$, shrinks boundary tables, limits the number of distinct destination PEs, and eliminates the power-of-2 constraint on PE count.

## 11.2 Priority 0 — Will Not Compile at Scale

**(1) `done_send_buf` grows with total PEs.** After sending data wavelets, each PE sends a "done" sentinel to every unique destination PE. The buffer that holds these sentinels (`done_send_buf`) is currently sized at `max_boundary`—which, under hash partitioning, can approach $P$ (the total PE count). At 900K PEs this alone would exceed the 48 KB SRAM budget. With locality-aware partitioning, the number of unique destination PEs drops to a small constant (typically <50), making a fixed-size buffer feasible.

**(2) `i16` types overflow at vertex counts above 32,767.** Loop indices and array lengths throughout the PE kernel (`pe_program.csl`) are declared as `i16`, which silently overflows when the global vertex count exceeds 32,767. The fix is mechanical: widen to `i32` everywhere.

**(3) Wavelet encoding has only 11 bits for destination PE.** The current encoding packs sender GID, destination PE, and color into a single 32-bit wavelet as [31:16] = sender_gid, [15:5] = dest_pe, [4:0] = color + 1. Eleven bits address at most 2,048 PEs. With locality-aware partitioning, the destination can be encoded as a relative ($dx$, $dy$) offset from the sender—6 bits each for modest hop distances—freeing the remaining bits for a wider color field or sender ID.

**(4) Layout generation uses a compile-time loop over all PEs.** `layout.csl` instantiates each PE via a CSL `comptime` loop. The compiler must unroll this loop entirely at compile time; at 900K iterations, compilation time and memory become untenable. CSL's `@set_tile_code` with a `.params` map can replace the per-PE loop with a single parameterized tile, letting the compiler generate one template and stamp it across the grid.

### 11.3 Priority 1 — Will Compile but Produce Wrong Results

**(5) No two-dimensional barrier.** The current barrier uses east-to-west row-reduce followed by west-to-east broadcast—a 1D scheme that only synchronizes within a single row. In a 2D grid, PEs in different rows can drift out of phase, leading to wavelet races between BSP rounds. A proper 2D barrier requires adding a column-wise reduction (south-to-north reduce, then north-to-south broadcast) after the row-wise pass.

**(6) Hash partitioning is the root cause of most scaling issues.** As discussed above, replacing `gid` & `pe_mask` with METIS partitioning is the foundational change. METIS minimizes edge cuts across partitions while balancing load, and a Hilbert-curve or strip-map assignment places neighboring partitions on physically adjacent PEs.

**(7) Relay buffers can overflow under heavy pass-through traffic.** Central PEs in the grid may have to relay wavelets from hundreds of crossing paths. Even with circular ring buffers, a fixed depth of `max_relay` cannot absorb arbitrarily large bursts. Locality-aware partitioning all but eliminates this problem by keeping most wavelets within a small neighborhood.

**(8) Uniform array sizing wastes SRAM on lightly loaded PEs.** Every PE allocates arrays for `max_local_verts` vertices regardless of how many it actually holds. Under hash partitioning, the maximum can be 3–5× the average. METIS balances partition sizes, bringing the max/avg ratio close to 1.0 and recovering significant SRAM headroom.

### 11.4 Priority 2 — Performance Bottlenecks

**(9) Linear search in `global_to_local`.** Every adjacency lookup during speculation walks the `global_vertex_ids` array linearly. At 50 local vertices with average degree 20, that is ∼50,000 scans per round per PE. If METIS assigns contiguous GID ranges, a direct index `gid` − `min_gid` gives $O(1)$ lookup.

**(10) Incoming-wavelet processing scans the full boundary table.** `process_incoming_wavelet` walks every boundary entry for each arriving wavelet, at cost $O(B \times R)$ per round. Sorting the boundary table by neighbor GID and using binary search reduces this to $O(R \log B)$.

**(11) Relay hops scale with grid dimension.** Each software-relay hop involves a full task activation cycle. On a 948×948 grid, worst-case path length is 1,896 hops. Locality-aware routing reduces this to typical 1–5 hops.

**(12) Division via repeated subtraction for 2D routing.** Converting a linear PE ID to $(row, col)$ uses a loop of $O(num_rows)$ subtractions. If the column count is a power of 2, a bit-shift gives the same result in $O(1)$. With relative offsets from locality-aware encoding, the division is unnecessary entirely.

### 11.5 Priority 3 — Host-Side Preprocessing

**(13) Sequential per-PE H2D transfers.** The host currently makes 12 blocking `memcpy_h2d` calls per PE. At 900K PEs, that is 10.8 million sequential API calls. Packing per-PE data into 3D tensors $[num_rows][num_cols][max_size]$ and issuing one rectangular transfer per symbol reduces the call count to ∼12.

**(14) Power-of-2 PE count constraint.** Hash partitioning requires `total_PEs` to be a power of two. The WSE-3 has ∼900K PEs, so the nearest valid count (524,288) wastes 42% of the fabric. METIS partitioning imposes no such constraint.

**(15) $O(n^2)$ conflict-graph construction.** The host builds the Pauli conflict graph by checking every pair of strings. For large inputs, this dominates preprocessing time. Vectorized numpy or the existing `graph_builder.py` CSR path are faster alternatives.

**(16) JSON serialization at scale.** Graph partition data for 900K PEs, serialized as JSON, can reach gigabytes. Switching to numpy `.npz` or HDF5 format offers 10–100× speed-up and far smaller files.

### 11.6 SRAM Budget After Locality-Aware Fixes

With METIS partitioning, ∼10-hop max relay distance, and ∼20 boundary edges on average, the per-PE memory budget fits comfortably:

| **Array** | **Bytes** | **Notes** |
|---|---:|---|
| `csr_offsets` | 204 | $51 \times 4$ |
| `csr_adj` | 2,000 | $500 \times 4$ |
| `colors` + `tentative` | 400 | $50 \times 4 \times 2$ |
| `global_vertex_ids` | 200 | $50 \times 4$ |
| `boundary_*` (3 arrays) | 600 | $50 \times 4 \times 3$ |
| `remote_recv_color` | 200 | $50 \times 4$ |
| `*_send_buf` (4 dirs) | 800 | $50 \times 4 \times 4$ |
| `relay_*_q` (4 dirs) | 1,024 | $64 \times 4 \times 4$ |
| Done tracking (fixed-size set) | 384 | $64 \times 6$ |
| `forbidden` | 128 | $32 \times 4$ |
| GID→local hash table | 1,024 | $128 \times 8$ |
| Misc (perf, timer, etc.) | 200 | |
| **Total** | **∼7.2 KB** | |

This leaves roughly 40.8 KB of headroom out of the 48 KB per-PE SRAM—ample room for larger local subgraphs or additional bookkeeping.

### 11.7 Recommended Implementation Order

The dependencies between these fixes suggest a natural sequence:

1. **Locality-aware partitioning** (#6): foundational change that reduces the severity of ten other issues.
2. **Widen types to `i32`** (#2): mechanical, required for compilation at scale.
3. **Fix layout compile-time loop** (#4): also required for compilation at scale.
4. **Redesign wavelet encoding** (#3): leverage locality for compact relative offsets.
5. **Replace `done_send_buf`** (#1): now feasible because locality limits the destination set.
6. **Bulk H2D transfers** (#13): independent host-side change, can proceed in parallel.
7. **Implement 2D barrier** (#5): correctness fix for full 2D mode.
8. **Optimize lookups** (#9, #10): performance polish, lowest priority.